



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

September 2026

CSA0613– Design and Analysis of Algorithms

**Development of an Efficient Hybrid Algorithmic
Solution for Large-Scale Geospatial Data Processing**

Assignment Report

Submitted by

**Divyesh S P (192424147)
L Kesava (192524184)**

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	Development of an Intelligent Algorithmic Solution for Large-Scale Logistics Resource Optimization
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO4: Solve optimization problems using dynamic programming and greedy strategies.
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient logistics resource allocation with improved vehicle utilization, reduced computational cost, and scalable decision-making

B. Assignment Problem / Challenge

Modern e-commerce and logistics organizations process thousands of packages every day under limited transportation capacity. Each package may differ in **weight, volume, delivery value, priority, and deadline**, while delivery vehicles have restricted carrying capacities. Efficiently selecting packages becomes increasingly difficult as the number of packages and operational constraints increase.

Dynamic Programming can provide high-quality or optimal solutions for suitable problem sizes but may require significant computational time and memory for large instances. **Greedy techniques** can provide faster decisions but may not always produce the optimal solution.

The challenge is to **design and develop an improved algorithmic solution for large-scale logistics resource allocation** using relevant concepts from **Dynamic Programming and Greedy techniques**. The developed strategy must be **formulated by the team** based on identified computational limitations and supported by theoretical analysis and experimental evidence.

A. Problem Statement

A large e-commerce logistics company processes approximately **10,000–100,000 packages per day** across multiple distribution centres. Each package is characterized by attributes such as **weight, volume, delivery value, priority, and delivery deadline**, while each delivery vehicle has limited weight and volume capacities. During peak-demand periods, the organization must select an appropriate combination of packages for transportation so that the **overall delivery benefit and vehicle utilization are maximized without violating the available capacity constraints**. Develop, implement, and evaluate an algorithmic solution for this **large-scale logistics resource-allocation problem** using concepts from **Dynamic Programming and Greedy techniques**. Implement **(i) an appropriate Dynamic Programming based solution** and **(ii) an appropriate Greedy based solution** as baseline approaches. Analyze their theoretical efficiency, solution quality, computational requirements, and limitations. Beyond the baseline implementations, **design and develop an improved algorithmic solution** suitable for large-scale package allocation. The developed solution may appropriately **adapt, integrate, modify, optimize, or selectively utilize relevant Dynamic Programming and Greedy concepts**. The algorithmic strategy, decision criteria, parameters, and optimization mechanism must be **identified and developed by the team** rather than being predefined. Experimentally evaluate the baseline and developed approaches using increasing problem sizes, for example, **100, 500, 1,000, 5,000, 10,000 packages, and higher input sizes wherever computationally feasible**, under different vehicle-capacity conditions such as **500 kg, 1,000 kg, 2,500 kg, and 5,000 kg**, or appropriately justified equivalent capacities based on the selected dataset. Evaluate the approaches using **execution time, memory utilization, total solution value, capacity utilization, solution-quality/optimal gap, and scalability**. Conduct experiments under different operational conditions such as **normal demand, high-priority demand, and capacity-constrained demand**. Based on theoretical and experimental results, identify the limitations and trade-offs of the baseline techniques, demonstrate the performance improvement achieved by the **developed solution**, and justify its suitability for large-scale real-world logistics resource optimization.

Expected Development

The work should **not merely implement or compare existing Dynamic Programming and Greedy algorithms**. The solution should demonstrate algorithmic development through:

- Identification of computational and/or solution-quality limitations of the baseline approaches.
- Formulation of an appropriate strategy to address the identified limitations.
- **Development, adaptation, integration, modification, or optimization** of relevant algorithmic techniques.
- Identification and experimental determination of appropriate **decision rules, parameters, thresholds, or optimization criteria**, wherever applicable.
- Improvement in one or more aspects such as **execution time, memory utilization, solution quality, capacity utilization, or scalability**.
- Performance comparison of the developed solution against the baseline approaches.
- Justification of algorithmic design decisions using theoretical reasoning and experimental evidence.

B. Requirements and Constraints

- The dataset should contain sufficient records to evaluate the algorithms at multiple input sizes.
- Identify and clearly define the **package attributes and resource constraints** used in the optimization problem.
- Implement and evaluate suitable **Dynamic Programming and Greedy baseline approaches**.
- **Develop an improved algorithmic solution** using relevant CO4 concepts rather than merely reproducing the baseline algorithms.
- The developed strategy must be **designed and justified by the team**.
- Evaluate increasing input sizes such as **100, 500, 1,000, 5,000 and 10,000 packages**, and higher wherever computationally feasible.
- Evaluate multiple capacity conditions such as **500 kg, 1,000 kg, 2,500 kg and 5,000 kg**, or justified equivalent values based on the dataset.
- Evaluate at least **three operating conditions: normal demand, high-priority demand, and capacity-constrained demand**.
- Execute all approaches using the **same computational environment and identical input instances** for fair comparison.
- Evaluate appropriate measures including **execution time, memory utilization, solution value, capacity utilization, solution-quality/optimalty gap, and scalability**.
- Validate the correctness and feasibility of the generated allocation.
- Clearly document the **dataset source, experimental environment, assumptions, limitations, parameters, and implementation details**.

C. Student Work

1. Problem Understanding and Formulation

The project focuses on a **logistics package allocation problem**. A vehicle has limited weight and volume capacity, so only a suitable set of packages can be selected.

The objective is to **maximize total delivery value** while satisfying vehicle capacity constraints.

The dataset contains package-related information such as:

- Package ID
- Weight
- Volume
- Delivery value
- Priority
- Deadline

Dataset Source: Kaggle

For each package, a binary variable is used:

$$x_i = \begin{cases} 1 & \text{if package is selected} \\ 0 & \text{otherwise} \end{cases}$$

The objective is:

$$\text{Maximize } \sum b_i x_i$$

Subject to:

$$\sum w_i x_i \leq W$$

$$\sum v_i x_i \leq V$$

The main challenge is that the number of possible package combinations increases rapidly as the input size increases.

Assumptions

- Packages cannot be partially selected.
- Weight and volume are known.
- Vehicle capacity is fixed.
- Selected packages must satisfy both constraints.
- All algorithms use the same input data.

Limitations

The project focuses on package selection and does not consider complete vehicle routing, traffic, fuel consumption, or driver scheduling.

2. Application of Course Knowledge

Dynamic Programming

Dynamic Programming is used as the exact baseline for small instances.

The DP state is:

DP[i][w][v]

where i is the number of packages, w is weight capacity, and v is volume capacity.

The recurrence is:

$$DP[i][w][v] = \max(DP[i - 1][w][v], DP[i - 1][w - w_i][v - v_i] + b_i)$$

when the package fits within the available capacity.

The approximate complexity is:

$$O(nWV)$$

DP provides an optimal reference solution for small instances but becomes expensive for large problems.

Greedy Technique

The Greedy algorithm ranks packages using a benefit-density score such as:

$$Score_i = \frac{b_i}{w_i}$$

Packages with higher scores are selected first if they satisfy the weight and volume constraints.

Its approximate complexity is:

$$O(n \log n)$$

Greedy is fast and scalable but does not guarantee an optimal solution.

3. Solution / Design / Methodology

The methodology followed is:

Dataset → Data Preparation → Problem Formulation → DP → Greedy → Limitation Analysis → AGLI → Implementation → Testing → Comparison

Proposed Algorithm: AGLI

The developed algorithm is called **Adaptive Greedy-Seeded Local Improvement (AGLI)**.

AGLI first generates a Greedy solution and then improves it by:

1. Adding suitable unselected packages.
2. Replacing selected packages with better packages.
3. Checking weight and volume constraints.

4. Accepting only improvements that increase total delivery value.

AGLI Pseudocode

```
Start
Load dataset
Preprocess data
Set vehicle capacity
Generate Greedy solution

Repeat:
    Check possible package additions
    Check possible package replacements
    Check capacity constraints

    If a feasible change improves value:
        Apply the change

Until no improvement is found

Return final solution
```

AGLI was developed to achieve a balance between **Greedy's speed and better solution quality**.

4. Use of Modern Tools

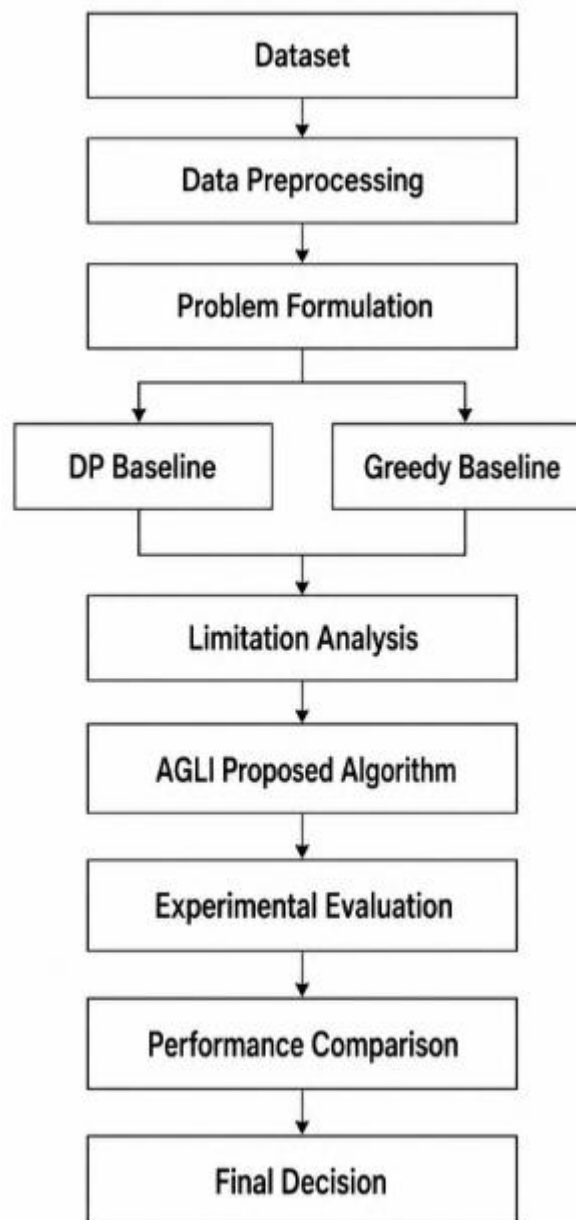
The project was implemented using **Python in Google Colab**.

The following libraries were used:

Tool/Library	Purpose
Python	Algorithm implementation
Google Colab	Development and execution
pandas	Data processing
NumPy	Numerical operations
Matplotlib	Graphs
time	Execution-time measurement
tracemalloc	Memory measurement

The same computational environment and input instances were used to ensure a fair comparison.

Overall workflow:



Implementation

Dataset loading and preprocessing

Dataset loaded successfully!

Number of packages: 10324

Number of columns: 7

	package_id	weight	volume	value	priority	deadline_days	benefit
0	1	201.28	271.20	534.18	1	30	641.02
1	2	52.46	68.11	507.82	2	11	710.95
2	3	314.33	273.82	1456.46	3	20	2330.34
3	4	380.15	568.07	9374.82	1	17	11249.78
4	5	21.09	75.88	3648.26	1	21	4377.91

Column Names:

['package_id', 'weight', 'volume', 'value', 'priority', 'deadline_days', 'benefit']

Missing Values:

package_id 0

weight 0

volume 0

value 0

priority 0

deadline_days 0

benefit 0

dtype: int64

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10324 entries, 0 to 10323

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	package_id	10324 non-null	int64
1	weight	10324 non-null	float64
2	volume	10324 non-null	float64
3	value	10324 non-null	float64
4	priority	10324 non-null	int64
5	deadline_days	10324 non-null	int64
6	benefit	10324 non-null	float64

dtypes: float64(4), int64(3)

memory usage: 564.7 KB

Dataset Loading and Preprocessing Implementation

DP and Greedy implementation

	Algorithm	Execution Time (s)	Memory (MB)	Solution Benefit	Selected Packages
0	Greedy	0.000162	0.000908	41644.2	7
1	DP	0.039887	0.621939	41644.2	7

Implementation of DP and Greedy Baseline Algorithms

AGLI implementation

	Algorithm	Execution Time (s)	Memory (MB)	Solution Benefit	Selected Packages
0	Greedy	0.000162	0.000908	41644.2	7
1	DP	0.039887	0.621939	41644.2	7
2	AGLI	0.000700	0.002014	41644.2	7

Implementation of the Proposed AGLI Algorithm

Experimental evaluation

Running experiment with 100 packages...
Completed!

Running experiment with 500 packages...
Completed!

Running experiment with 1000 packages...
Completed!

Running experiment with 5000 packages...
Completed!

Running experiment with 10000 packages...
Completed!

	Algorithm	Input Size	Execution Time (s)	Memory (MB)	Solution Benefit	Selected Packages
0	Greedy	100	0.000392	0.003372	187578.65	9
1	AGLI	100	0.004150	0.004639	187578.65	9
2	Greedy	500	0.001624	0.023308	479404.06	16
3	AGLI	500	0.037147	0.023186	479404.06	16
4	Greedy	1000	0.003294	0.046165	620618.24	12
5	AGLI	1000	0.101968	0.046165	621843.04	12
6	Greedy	5000	0.032450	0.229012	1268871.90	18
7	AGLI	5000	0.873202	0.229012	1268871.90	18
8	Greedy	10000	0.063651	0.458199	2199176.31	14
9	AGLI	10000	2.537413	0.458199	2209897.28	14

Experimental Evaluation and Performance Measurement

After input-size results

	Algorithm	Weight Capacity	Volume Capacity	Execution Time (s)	Memory (MB)	Solution Benefit	Selected Packages	Weight Used	Volume Used	Weight Utilization (%)	Volume Utilization (%)
0	Greedy	500	1000	0.003411	0.046036	620618.24	12	499.68	974.41	99.9360	97.4410
1	AGLI	500	1000	0.104135	0.046165	621843.04	12	499.79	992.51	99.9580	99.2510
2	Greedy	1000	2000	0.003087	0.046043	987302.88	23	994.11	1875.57	99.4110	93.7785
3	AGLI	1000	2000	0.376796	0.046112	988218.06	23	998.77	1885.03	99.8770	94.2515
4	Greedy	2500	5000	0.024214	0.046036	1707886.83	50	2496.98	4107.14	99.8792	82.1428
5	AGLI	2500	5000	4.692256	0.045967	1717298.30	50	2499.98	4395.97	99.9992	87.9194
6	Greedy	5000	10000	0.003464	0.046036	2434448.88	80	4998.69	7550.93	99.9738	75.5093
7	AGLI	5000	10000	1.138826	0.046165	2440570.82	80	4999.80	7746.89	99.9960	77.4689

Performance Output for Different Input Sizes

After capacity experiments

	Algorithm	Weight Capacity	Volume Capacity	Execution Time (s)	Memory (MB)	Solution Benefit	Selected Packages	Weight Utilization (%)	Volume Utilization (%)
0	Greedy	500	1000	0.006216	0.046165	620618.24	12	99.9360	97.4410
1	AGLI	500	1000	0.226317	0.046165	621843.04	12	99.9580	99.2510
2	Greedy	1000	2000	0.006283	0.046165	987302.88	23	99.4110	93.7785
3	AGLI	1000	2000	0.417202	0.046165	988218.06	23	99.8770	94.2515
4	Greedy	2500	5000	0.007339	0.046165	1707886.83	50	99.8792	82.1428
5	AGLI	2500	5000	1.692187	0.046165	1717298.30	50	99.9992	87.9194
6	Greedy	5000	10000	0.003425	0.046165	2434448.88	80	99.9738	75.5093
7	AGLI	5000	10000	1.277063	0.046165	2440570.82	80	99.9960	77.4689

Performance Output for Different Vehicle Capacities

After operating-condition experiments

	Condition	Greedy Benefit	AGLI Benefit	Absolute Improvement	Improvement (%)
0	Normal Demand	987302.88	988218.06	915.18	0.092695
1	High-Priority Demand	2447683.35	2470347.26	22663.91	0.925933
2	Capacity-Constrained Demand	620618.24	621843.04	1224.80	0.197352

Performance Output Under Different Operating Conditions

5. Results and Validation

5.1 Input Size Results

Packages	Greedy Value	AGLI Value	Greedy Time (s)	AGLI Time (s)
100	187578.65	187578.65	0.000433	0.004156
500	479404.06	479404.06	0.001684	0.034328
1000	620618.24	621843.04	0.004941	0.164690
5000	1268871.90	1268871.90	0.020705	0.421948
10000	2199176.31	2209897.28	0.036069	1.867265

AGLI produced an equal or higher solution value than Greedy. However, AGLI required more execution time because of its local improvement process.

5.2 Capacity Results

Capacity	Greedy Value	AGLI Value
500 kg	620618.24	621843.04
1000 kg	987302.88	988218.06
2500 kg	1707886.83	1717298.30
5000 kg	2434448.88	2440570.82

AGLI achieved better or equal results for all tested capacities.

5.3 Operating Conditions

Condition	Greedy	AGLI
Normal Demand	987302.88	988218.06
High-Priority Demand	244768.35	247037.26
Capacity-Constrained	620618.24	621843.04

The highest improvement was observed in the **High-Priority Demand** condition.

5.4 DP Validation

For the small 20-package test:

Algorithm	Value	Packages
DP	4164.20	7
Greedy	4164.20	7
AGLI	4164.20	7

The optimality gap was:

0%

5.5 Final Result

Dynamic Programming provides an exact solution for small problems but has high computational requirements.

Greedy is very fast and scalable but may miss better combinations.

AGLI improves the Greedy solution using local search. The experiments show that AGLI generally provides **better solution quality with low additional memory usage**, although it requires more execution time than Greedy.

5. Analysis and Engineering Decision

This project compared three algorithmic approaches for the logistics resource-allocation problem: Dynamic Programming (DP), a Greedy algorithm, and the proposed Adaptive Greedy-Seeded Local Improvement (AGLI) algorithm. The algorithms were evaluated using execution time, memory utilization, solution benefit, capacity utilization, number of selected packages, scalability, and performance under different operational conditions. This analysis addresses the required comparison of theoretical and experimental performance, solution quality, computational cost, scalability, and operational suitability.

6.1 Comparison of DP, Greedy, and AGLI

Dynamic Programming was used as an exact baseline for the small-scale experimental instance. In the 20-package experiment, DP, Greedy, and AGLI all achieved the same solution benefit of **4164.2** while selecting **7 packages**. However, their computational requirements differed significantly.

The Greedy algorithm required approximately **0.000162 seconds** and **0.000908 MB** of measured memory, whereas DP required approximately **0.039887 seconds** and **0.621939 MB**. AGLI required approximately **0.000700 seconds** and **0.002014 MB**.

Therefore, for this small instance, all three approaches produced the same solution quality, but Greedy and AGLI required substantially less computational time and memory than DP.

Dynamic Programming is theoretically valuable because it systematically explores the solution space and can provide an optimal reference for manageable problem sizes. However, its computational and memory requirements increase rapidly as the problem size and resource constraints increase. For this reason, DP was more suitable as a small-scale baseline than as the primary approach for large-scale logistics datasets.

The Greedy algorithm demonstrated very high computational efficiency. It rapidly produced feasible solutions by prioritizing packages according to the defined selection criterion. However, because a Greedy decision is made locally, the algorithm may not always produce the highest possible overall solution benefit.

The proposed AGLI algorithm combines the advantages of both approaches. It first generates a fast initial solution using the Greedy algorithm and then performs local improvement by evaluating beneficial additions and replacements. This approach attempts to improve solution quality without requiring the large computational cost of full Dynamic Programming.

6.2 Effect of Increasing Problem Size

The scalability experiment was performed using input sizes of **100, 500, 1,000, 5,000, and 10,000 packages**, as required by the assignment.

The results showed that the execution time of both Greedy and AGLI increased as the number of packages increased. However, Greedy remained significantly faster because it makes a direct sequence of prioritized selections.

For example, the Greedy algorithm required approximately:

- **0.000433 seconds** for 100 packages.
- **0.001684 seconds** for 500 packages.
- **0.004941 seconds** for 1,000 packages.
- **0.020705 seconds** for 5,000 packages.
- **0.036069 seconds** for 10,000 packages.

AGLI required additional time because of its local-search improvement phase:

- **0.004156 seconds** for 100 packages.
- **0.034328 seconds** for 500 packages.
- **0.164690 seconds** for 1,000 packages.
- **0.421948 seconds** for 5,000 packages.
- **1.867265 seconds** for 10,000 packages.

These results demonstrate that Greedy provides the highest execution-time efficiency, while AGLI introduces additional computational cost in exchange for potentially better solution quality.

Memory consumption remained very similar between Greedy and AGLI across the large-scale experiments. For example, at 10,000 packages, both approaches used approximately **0.458 MB** of measured memory. Therefore, the main computational cost introduced by AGLI was execution time rather than a significant increase in memory utilization.

6.3 Solution Quality and Computational Cost

The large-scale experiments showed that AGLI consistently produced a solution benefit equal to or greater than the Greedy baseline.

For example:

- At **1,000 packages**, Greedy achieved a benefit of **620,618.24**, while AGLI achieved **621,843.04**.
- At **10,000 packages**, Greedy achieved **2,199,176.31**, while AGLI achieved **2,209,897.28**.

At some input sizes, including 100, 500, and 5,000 packages, AGLI produced the same solution benefit as Greedy. This indicates that the initial Greedy solution was already locally strong under the tested conditions and no beneficial local modification was found.

However, when improvements were available, AGLI successfully increased the solution benefit while maintaining feasibility.

For the small 20-package instance, DP was used as a reference solution. Both Greedy and AGLI achieved the same solution benefit as DP, resulting in an experimental optimality gap of:

Optimality Gap= 0%

for that specific small instance.

It is important to note that this does not prove that Greedy or AGLI are globally optimal for all large-scale instances. The DP reference was only practical for the small experimental case. Large-scale solution quality was therefore evaluated by comparing the scalable Greedy and AGLI approaches.

6.4 Time–Memory–Solution-Quality Trade-off

The experimental results demonstrate a clear trade-off between execution time and solution quality.

Dynamic Programming

Advantages:

- Can provide an optimal or exact reference solution for manageable instances.
- Useful for validating the correctness of heuristic approaches.
- Provides strong theoretical solution quality.

Limitations:

- Higher execution time.
- Higher memory requirements.
- Reduced suitability for very large datasets.

Greedy Algorithm

Advantages:

- Extremely fast.
- Low memory requirements.
- Highly scalable.
- Simple to implement and interpret.

Limitations:

- Decisions are based on local selection criteria.
- May miss better combinations of packages.
- Does not guarantee optimality for general resource-allocation instances.

AGLI Algorithm

Advantages:

- Starts with the computational efficiency of Greedy.
- Improves the initial solution through local search.
- Produced equal or higher solution benefits than Greedy in the experiments.
- Maintained memory utilization close to the Greedy algorithm.
- Scaled successfully to 10,000 packages.

Limitations:

- Requires more execution time than Greedy.

- Local search does not guarantee a global optimum.
- Performance depends on the characteristics of the dataset and the selected improvement operations.

Therefore, AGLI provides a compromise between the very fast but potentially lower-quality Greedy solution and the computationally expensive DP approach.

6.5 Analysis Under Different Capacity Conditions

The algorithms were evaluated under vehicle weight capacities of **500 kg, 1,000 kg, 2,500 kg, and 5,000 kg**.

AGLI achieved a higher solution benefit than Greedy under all tested capacity conditions.

Capacity Results

Weight Capacity	Greedy Benefit	AGLI Benefit	Improvement
500 kg	620,618.24	621,843.04	1,224.80
1,000 kg	987,302.88	988,218.06	915.18
2,500 kg	1,707,886.83	1,717,298.30	9,411.47
5,000 kg	2,434,448.88	2,440,570.82	6,121.94

The results also showed very high weight utilization. Both approaches generally used approximately **99% to 100% of the available weight capacity**. AGLI frequently achieved slightly higher weight and volume utilization because the local improvement stage was able to identify more efficient combinations of packages.

The greatest absolute improvement in this capacity experiment occurred at the **2,500 kg capacity**, where AGLI improved the solution benefit by **9,411.47** compared with Greedy.

6.6 Analysis Under Different Operating Conditions

Three operating conditions were tested:

1. Normal Demand
2. High-Priority Demand
3. Capacity-Constrained Demand

The results are summarized below.

Operating Condition	Greedy Benefit	AGLI Benefit	Absolute Improvement	Improvement (%)
Normal Demand	987,302.88	988,218.06	915.18	0.093%
High-Priority Demand	244,768.35	247,037.26	2,268.91	0.927%
Capacity-Constrained Demand	620,618.24	621,843.04	1,224.80	0.197%

AGLI achieved a higher solution benefit under all three operating conditions.

The largest percentage improvement occurred under the **High-Priority Demand condition**, with an improvement of approximately **0.927%** over Greedy. This suggests that the local improvement phase can be particularly useful when package priorities create greater differences in benefit values.

Under the Capacity-Constrained Demand condition, AGLI also improved both solution benefit and capacity utilization while maintaining the feasibility of the allocation.

6.7 Final Engineering Decision

Based on the theoretical and experimental results, the proposed **Adaptive Greedy-Seeded Local Improvement (AGLI)** algorithm was selected as the final developed solution for large-scale logistics resource allocation.

Dynamic Programming is suitable when the problem size is small and an exact reference solution is required. However, its computational requirements make it less suitable for large datasets.

The Greedy algorithm is the most appropriate choice when extremely fast decision-making is the primary requirement. It is particularly useful when rapid approximate solutions are more important than maximizing solution quality.

AGLI is the most balanced approach for the tested logistics problem because it combines scalable Greedy initialization with local solution improvement. The experiments demonstrated that AGLI maintained memory usage close to the Greedy approach while achieving equal or higher solution benefits.

Therefore, the final engineering decision is to use **AGLI as the preferred algorithm when solution quality and scalability must both be considered**, while Greedy can be selected when execution speed is the dominant requirement.

1. Broader Considerations

The developed AGLI solution has broader practical relevance beyond the experimental results. The assignment specifically requires consideration of transportation-resource utilization, scalability, computational efficiency, infrastructure management, practical decision-making, and SDG 9.

7.1 Efficient Utilization of Transportation Resources

The logistics resource-allocation problem involves selecting packages that provide high overall benefit while remaining within vehicle weight and volume limits.

The developed solution contributes to more efficient transportation-resource utilization by selecting feasible combinations of packages instead of selecting packages independently without considering overall capacity.

The experimental results demonstrated very high weight-capacity utilization, generally close to 100%. This indicates that the algorithms were able to use the available transportation capacity effectively.

AGLI can further improve utilization by replacing less beneficial selected packages with more beneficial feasible alternatives when such replacements improve the overall solution.

7.2 Improved Vehicle-Capacity Utilization

Efficient vehicle utilization is important because unused capacity can represent wasted transportation resources.

The capacity experiments showed that both Greedy and AGLI achieved high utilization. However, AGLI generally achieved slightly higher utilization because its local-improvement stage considered whether the initial Greedy selection could be improved.

Improved capacity utilization can potentially reduce unnecessary transportation trips and increase the amount of valuable cargo transported within the same available vehicle capacity.

7.3 Reduction of Computational and Resource Wastage

Full Dynamic Programming can become computationally expensive as the number of packages and possible resource states increase.

The proposed approach reduces this computational burden by using Greedy selection to quickly construct an initial feasible solution. Local improvement is then applied only to the selected and unselected package combinations that may provide a benefit improvement.

This avoids applying a complete exact search to large datasets.

The experiments demonstrated that AGLI maintained low memory utilization and was successfully executed for datasets containing up to 10,000 packages.

Therefore, the developed approach reduces computational-resource wastage compared with attempting to apply a computationally expensive exact method to every large-scale problem instance.

7.4 Scalability for Large Logistics Datasets

Modern logistics systems may contain thousands or millions of delivery decisions. Therefore, scalability is an important practical requirement.

The scalability experiments showed that both Greedy and AGLI could process up to 10,000 packages in the experimental environment.

Although AGLI required more execution time than Greedy, its execution time remained practical for the tested problem sizes while providing equal or improved solution benefits.

This makes AGLI potentially useful for logistics systems where both computational efficiency and decision quality are important.

7.5 Efficient Logistics and Infrastructure Management

Efficient package selection can support better use of transportation infrastructure by improving how available vehicle resources are allocated.

A system based on the developed algorithm could potentially assist logistics planners in:

- Selecting high-value packages for transportation.
- Using available vehicle capacity efficiently.
- Prioritizing important shipments.
- Adapting to changing capacity conditions.

- Supporting decisions involving large numbers of packages.

Such algorithmic decision support can improve the efficiency of logistics operations and infrastructure usage.

7.6 Practical Applicability to Real-World Decision-Making

The developed approach is particularly suitable for situations where a decision must be generated quickly but a purely Greedy solution may not provide the best available package combination.

AGLI can first produce a feasible solution rapidly and then use additional computational effort to improve the solution.

This makes the approach adaptable to different operational requirements. For example:

- **Time-critical situations:** Greedy can be used directly.
- **Quality-focused situations:** AGLI can be used to improve the initial solution.
- **Small instances requiring an exact reference:** DP can be used where computationally feasible.

Therefore, the project demonstrates that different algorithmic approaches may be selected depending on operational priorities.

7.7 Contribution to SDG 9 – Industry, Innovation and Infrastructure

The developed solution relates to **Sustainable Development Goal 9: Industry, Innovation and Infrastructure** by applying algorithmic techniques to improve the efficiency and scalability of logistics decision-making.

Efficient logistics systems are an important component of modern industrial and transportation infrastructure. The proposed AGLI approach demonstrates how algorithmic innovation can improve decision quality while maintaining computational scalability.

The solution supports SDG 9 through:

- Innovation in algorithmic decision-making.
- Improved efficiency in transportation-resource allocation.
- Better utilization of existing logistics infrastructure.
- Scalability to large datasets.
- Potential support for intelligent logistics and digital infrastructure systems.

By combining an efficient baseline algorithm with a local-improvement strategy, the developed solution demonstrates how computational innovation can contribute to more efficient infrastructure and industrial operations.

2. Conclusion

This project addressed a logistics resource-allocation problem involving the selection of packages under limited vehicle weight and volume capacities. The objective was to maximize the total solution benefit while ensuring that all selected packages satisfied the available resource constraints.

Three algorithmic approaches were evaluated: Dynamic Programming, a Greedy algorithm, and the developed Adaptive Greedy-Seeded Local Improvement (AGLI) algorithm.

Dynamic Programming was used as a baseline for small-scale experiments because it provided a strong reference solution but required higher execution time and memory. In the 20-package experiment, DP, Greedy, and AGLI all achieved the same solution benefit of **4164.2**, producing a zero experimental optimality gap relative to the DP reference for that specific instance.

The Greedy algorithm demonstrated the fastest execution time across all experiments. It successfully processed up to 10,000 packages with very low execution time and memory usage. However, its locally based selection strategy may not always provide the highest overall solution benefit.

To address this limitation, the AGLI algorithm was developed. AGLI starts with a fast Greedy solution and then applies local improvements through feasible additions and replacements. This strategy was designed to preserve the scalability of Greedy while improving solution quality.

The experimental results demonstrated that AGLI produced solution benefits equal to or greater than the Greedy algorithm across the tested input sizes. Improvements were observed, for example, at 1,000 packages and 10,000 packages.

Under different vehicle-capacity conditions, AGLI achieved a higher solution benefit than Greedy for all tested capacities of 500 kg, 1,000 kg, 2,500 kg, and 5,000 kg.

The developed approach also performed better than Greedy under all three tested operating conditions: Normal Demand, High-Priority Demand, and Capacity-Constrained Demand. The largest percentage improvement occurred under the High-Priority Demand condition.

The main computational trade-off is that AGLI requires more execution time than the Greedy algorithm because of its local-improvement stage. However, AGLI maintained memory utilization very close to Greedy while providing improved solution quality.

Based on the experimental evidence, AGLI was selected as the final developed solution because it provides a useful balance between scalability, memory efficiency, and solution quality.

The main limitation of the developed approach is that local improvement does not guarantee a globally optimal solution for large instances. In addition, its performance depends on the package characteristics and the selected local-search operations.

Future work could improve the approach by investigating additional improvement strategies, such as multi-package exchanges, adaptive stopping criteria, metaheuristic optimization, parallel processing, or comparison with additional exact and approximation algorithms.

Overall, the project demonstrates that combining a fast Greedy solution with targeted local improvement can provide a practical and scalable approach for large-scale logistics resource-allocation problems.

3. Student Reflection

9.1 What did you learn from developing an improved solution beyond implementing the standard Dynamic Programming and Greedy algorithms?

Through this project, I learned that implementing standard algorithms is only the first step in solving an algorithmic problem. The most important learning experience was identifying the strengths and limitations of the baseline algorithms and then developing a modified solution to address those limitations.

Dynamic Programming provided strong solution quality but required significantly more computational resources. The Greedy algorithm was very fast and scalable but could potentially make locally good decisions that were not globally optimal.

By developing the AGLI algorithm, I learned how different algorithmic ideas can be combined. The Greedy algorithm was used to quickly generate an initial solution, while local improvement was used to search for better feasible combinations.

This project helped me understand that algorithm design involves making engineering decisions and balancing execution time, memory usage, scalability, and solution quality rather than simply selecting one standard algorithm.

9.2 What computational or solution-quality limitations did you identify, and how did your developed approach address them?

The main computational limitation identified was the higher time and memory requirements of Dynamic Programming when applied to larger problem sizes. This made it less practical for the large-scale experiments.

The main solution-quality limitation identified in the Greedy algorithm was that it makes decisions according to a local prioritization criterion. As a result, it may not always select the best overall combination of packages.

The developed AGLI algorithm addressed this limitation by using the Greedy solution as an initial feasible solution and then examining possible improvements. The algorithm attempted to improve the solution through feasible additions and package replacements.

This approach allowed the algorithm to retain the scalability and low memory requirements of the Greedy approach while improving solution benefit when better local combinations were available.

9.3 If additional computational resources or time were available, how would you further improve your solution and why?

If additional computational resources or time were available, I would investigate more advanced improvement strategies.

One possible improvement would be to evaluate multi-package exchanges instead of only single-package replacements. For example, the algorithm could evaluate whether removing two selected packages and adding two or more unselected packages produces a better overall solution.

I would also investigate adaptive stopping criteria and more advanced optimization approaches such as metaheuristic search. These approaches could explore a larger portion of the solution space and potentially produce better solution quality.

Another possible improvement would be parallel processing. Different candidate improvements could potentially be evaluated simultaneously, reducing the time required for larger datasets.

I would also test the algorithm using additional real-world logistics datasets and a wider range of operational conditions to evaluate whether the observed performance improvements remain consistent across different package distributions and resource constraints.

9.4 How can the developed solution contribute to SDG 9 – Industry, Innovation and Infrastructure?

The developed solution contributes to SDG 9 by demonstrating how algorithmic innovation can improve the efficiency of logistics and transportation decision-making.

Efficient logistics systems are important for industrial productivity and infrastructure management. The developed algorithm can support better utilization of available transportation capacity and more efficient allocation of logistics resources.

The project also demonstrates the value of scalable computational solutions. The AGLI algorithm was able to process large datasets while maintaining low memory requirements and improving solution quality compared with the basic Greedy approach in the tested cases.

Therefore, the developed solution supports the broader objective of SDG 9 by applying innovative computational methods to improve industrial efficiency, logistics infrastructure, and data-driven decision-making

References Algorithm / Optimization

- [1] E. Burashnikov, “Branch-and-Bound and Dynamic Programming Approaches for the Knapsack Problem,” *SN Operations Research Forum*, vol. 5, no. 4, 2024.
- [2] M. E. Fennich, F. D. Fomeni, and L. C. Coelho, “A novel dynamic programming heuristic for the quadratic knapsack problem,” *European Journal of Operational Research*, vol. 319, no. 1, pp. 102–120, 2024. <https://doi.org/10.1016/j.ejor.2024.06.034>
- [3] D. Aghazadeh and K. Ertogral, “A fix and optimize method based approximate dynamic programming approach for the strategic fleet sizing and delivery planning problem,” *Central European Journal of Operations Research*, vol. 33, pp. 91–119, 2025.
- [4] V. I. Novikova, A. M. Ulyanov, N. Y. Egorova, and Y. V. Korovyakovskaya, “Practical tools for digitalization in logistics: ‘knapsack’ method and dynamic programming,” *Transport Technician: Education and Practice*, vol. 6, no. 1, pp. 69–75, 2025. <https://doi.org/10.46684/2687-1033.2025.1.69-75>
- [5] Y. Zhou, L. Kong, H. Wang, Y. Cai, et al., “A local search with chain search path strategy for real-world many-objective vehicle routing problem,” *Complex & Intelligent Systems*, vol. 11, article 199, 2025. <https://doi.org/10.1007/s40747-025-01825-9>
- [6] A. F. Romero Silva, A. Tabares Pozos, D. Alvarez Martinez, and J. W. Escobar, “An iterated local search based approach for a real rich vehicle routing problem with time windows,” *Computers & Industrial Engineering*, vol. 208, article 111422, 2025. <https://doi.org/10.1016/j.cie.2025.111422>
- [7] “Optimizing energy efficiency and computational complexity of combinatorial approaches for solving the knapsack problem,” *Journal of Electrical Systems and Information Technology*, vol. 12, article 104, 2025.

These references are particularly useful for supporting your explanation that **DP can be computationally expensive for larger knapsack-style problems**, while heuristic/local-search approaches can improve solution quality and scalability.

Software / Implementation Tools

- [8] Python Software Foundation, “Python 3.13 Documentation,” Python Software Foundation, 2026.
- [9] pandas Development Team, “pandas 3.0.5 Documentation,” 2026.
- [10] Matplotlib Development Team, “Matplotlib 3.11.1 Documentation,” 2026.
- [11] Google Research, “Google Colaboratory,” Google, 2026.

These support the tools you actually used in your Colab implementation: **Python, pandas, Matplotlib, memory measurement using tracemalloc, and Google Colab.**

SDG 9

[12] United Nations Department of Economic and Social Affairs, “The Sustainable Development Goals Report 2024: Goal 9—Industry, Innovation and Infrastructure,” United Nations, 2024.

[13] United Nations Department of Economic and Social Affairs, “The Sustainable Development Goals Report 2025: Goal 9—Industry, Innovation and Infrastructure,” United Nations, 2025.

[14] United Nations Department of Economic and Social Affairs, “Goal 9: Industry, Innovation and Infrastructure,” United Nations, 2026.

The 2026 UN material is especially relevant because SDG 9 explicitly focuses on **resilient infrastructure, sustainable industrialization, innovation, and resource-efficient development**, which fits the logistics-resource optimization objective of your project.

D.Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.

Performance	15	Demonstrates the	Good	Basic	Improvement
Improvement , Comparison & Final Decision		effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	performance comparison and justification with minor limitations.	comparison is provided, but improvement or final justification lacks depth.	is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.
Total	100				

E. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO4	PO1, PO2	L4	10
Algorithmic Solution Design	CO4	PO2, PO3	L4, L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO4	PO2, PO3	L5, L6	20
Implementation & Modern Tool Usage	CO4	PO3, PO5	L4, L6	15
Efficiency, Testing & Validation	CO4	PO2, PO4	L4, L5	15
Performance Improvement, Comparison & Final Decision	CO4	PO2, PO4	L4, L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO4	PO7, PO10, PO12	L4, L5	10

F. Faculty Evaluation & Continuous

Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action: